



University of sigmas

# Oiowe chady

Adam Biel, Mateusz Jurach, Karol Żeleźnik

2024-03-07

## 1    Templatki

## 2    Struktury danych

## 3    Optymalizacje

# Templatki (1)

### .bashrc

11 lines

```
c() {
    g++ -std=c++20 -Wall -Wno-sign-conversion -Wextra -Wshadow -
        Wconversion -Wfloat-equal \
        -fsanitize=address,undefined -DLOCAL -DDEBUG -D_GLIBCXX_DEBUG
        -ggdb3 $1.cpp -o $1
}

nc() {
    g++ -std=c++20 -DLOCAL -O3 -static $1.cpp -o $1
}

alias cp='cp -i'
alias mv='mv -i'
```

### .vimrc

7 lines

```
set nocompatible
filetype indent on
set autoindent
set number
syntax on
set ts=4 sw=4
set rnu
```

### main.cpp

23 lines

```
#include <bits/stdc++.h>
using namespace std;
#ifdef DEBUG
auto&operator<<(auto &o, pair<auto, auto> p) {o << "(" << p.
    first << ", " << p.second << ")"; return o;}
auto operator<<(auto &o, auto x)->decltype(x.end(), o) {o<<"{";
    for(auto e : x) o<<e<<" "; return o<<"}";}
#define debug(X) cerr << "["#X"]": " << X << '\n';
#else
#define cerr if(0)cout
#define debug(X) ;
#endif
#define ll long long
#define all(v) (v).begin(), (v).end()
#define ssize(x) int(x.size())
#define fi first
#define se second
#define mp make_pair
#define eb emplace_back

int main() {
    ios_base::sync_with_stdio(false); cin.tie(nullptr);

    return 0;
}
```

### gen.cpp

Opis: Dodatek do generowania testów, lub innego losowania. Ziarno można zmieniać np. rng.seed(atoi(argv[1]));

```
mt19937 rng(random_device{}());
```

```
int rd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}
```

# Struktury danych (2)

### wavelet.cpp

Opis:  $O(\log^2 \text{ALPH})$  czasowo,  $O(n \log \text{ALPH})$  pamięciowo. Constructor przyjmuje ciąg na którym rozpinamy drzewo, rank(l, r, k) zwraca  $k$ -tą najmniejszą liczbę na przedziale  $[l; r]$ . (**Uwaga!**) pamięć około 262 Mb, dla  $N, Q \leq 2 \cdot 10^5$ .

```
//funkcja cnt jest w  $O(\log)$  co daje cale operacje w  $O(\log^2)$ ,
//da sie cnt w czasie stalym — do dopisania
struct wavelet_tree {
    const static int ALPH = (1<<18)-1;
    struct Node {
        int low, high, mid;
        vector<int> v;
        Node *left, *right;

        Node() {
            low = 0, high = ALPH;
            left = right = nullptr;
            mid = (low+high)/2;
        }

        Node (const vector<int> &x, const vector<int> &t, int a =
            0, int b = ALPH): low(a), high(b), v(x) {
            mid = ((ll)low+high)/2;
            if(ssize(x) && low != high) {
                vector<int> div[2];
                for(auto e : v) div[t[e]<=mid].eb(e);
                if(ssize(div[1])) left = new Node(div[1], t, low, mid);
                else left = nullptr;
                if(ssize(div[0])) right = new Node(div[0], t, mid+1,
                    high);
                else right = nullptr;
            }
            else {
                left = right = nullptr;
            }
        }
    };

    Node *root;
    wavelet_tree(const vector<int> &b) {
        vector<int> idx;
        REP(i, ssize(b)) idx.eb(i);
        root = new Node(idx, b);
    };

    int cnt(Node *v, int l, int r) {
        if(v == nullptr) return 0;
        return int(upper_bound(all(v->v), r)-lower_bound(all(v->v),
            l));
    }

    int rank(int l, int r, int k, Node *v) {
        if(v->low == v->high) return v->low;
        if(cnt(v->left, l, r) >= k) return rank(l, r, k, v->left);
        else return rank(l, r, k-cnt(v->left, l, r), v->right);
    }

    int rank(int l, int r, int k) {
        return rank(l, r, k, root);
    }

    void clear(Node *v) {
```

```
        if(v->left != nullptr) clear(v->left);
        if(v->right != nullptr) clear(v->right);
        delete v;
    }
    ~wavelet_tree() {
        clear(root);
    }
};
```

### fenwick-tree.cpp

Opis: (Binary Indexed Tree)  $O(\log N)$ , indeksowane od 0. Update w punkcie, zapytanie na przedziale

```
struct BIT {
    vector<int> tree;
    int N;
    BIT() {N = 0};
    BIT(int n) {
        N = n;
        tree.resize(N);
    }
    void resize(int n) {
        N = n;
        tree.resize(N);
    }
    void update(int p, int x) {
        for (++p; p <= N; p += (p&-p)) tree[p-1] += x;
    }
    int sum(int r) {
        int res = 0;
        for (; r > 0; r -= (r&-r)) res += tree[r-1];
        return res;
    }
    int query(int l, int r) {return sum(r+1)-sum(l);}
    int lower_bound(int sum) {
        int pos = 0;
        for (int pw = 1<<25; pw; pw>=1) {
            int npos = pos+pw;
            if (npos <= N && tree[npos-1] < sum) {
                pos = npos; sum -= tree[pos-1];
            }
        }
        return pos;
    }
};
```

# Optymalizacje (3)

## dodać flushowanie bufora i zmienić nazwy buf (buf-in, buf-out)

## 3.1    FastOut

Jest kilka opcji. Różnią się długością (i złożonością kodu), bądź używaniem funkcji write() z bibliotekiunistd.h (gdyby był z nią problem to można użyć opcji bez tych funkcji).

Do każdego kodu dopisany jest przykładowy czas (wypisanie  $10^7$  liczb). Rozmiary bufora zależą od rozmiaru wyjścia (raczej mniejszy niż w poniższych kodach). Rozmiar powinien uwzględnić każdą cyfrę i spację, a w pliku fast.cpp dodatkowe spacje wypisywane na początku liczby (maksymalnie 4 na liczbę). Jeżeli wyjście jest za duże, to bufor może być mniejszy, a kod podzielony na kilka części oddzielonych flushowaniem (ważne by kod podzielić, **nie** dodawać ifa w środku żadnej funkcji, bo będzie wolno).

fastout.cpp

Opis: czas: ok. 0,067s na time, 195ms na oiejq.

```
char buf[200 * 1'024 * 1'024 + 100'000*sizeof(unsigned long)];
char *buf_ptr = buf;

void write_char(char c) {
    *buf_ptr++ = c;
}

constexpr unsigned long
build_long(char c1, char c2, char c3, char c4, char c5, char c6
, char c7, char c8) {
    unsigned long n = c8;
    n <= 8;
    n |= c7;
    n <= 8;
    n |= c6;
    n <= 8;
    n |= c5;
    n <= 8;
    n |= c4;
    n <= 8;
    n |= c3;
    n <= 8;
    n |= c2;
    n <= 8;
    n |= c1;
    return n;
}

template <bool LeadingZeroes> struct output {

    constexpr output() {
        char lead = LeadingZeroes ? '0' : ' ';

        for(unsigned long c1 = 1; c1 <= 9; ++c1)
            for(unsigned long c2 = 0; c2 <= 9; ++c2)
                for(unsigned long c3 = 0; c3 <= 9; ++c3)
                    for(unsigned long c4 = 0; c4 <= 9; ++c4)
                        for(unsigned long c5 = 0; c5 <= 9; ++c5)
                            arr[10'000*c1+1'000*c2+100*c3+10*c4+c5] =
                                build_long('0'+c1, '0'+c2, '0'+c3, '0'+c4, '
                                0'+c5, '\n', '0', '0');

        for(unsigned long c2 = 1; c2 <= 9; ++c2)
            for(unsigned long c3 = 0; c3 <= 9; ++c3)
                for(unsigned long c4 = 0; c4 <= 9; ++c4)
                    for(unsigned long c5 = 0; c5 <= 9; ++c5)
                        arr[1'000*c2+100*c3+10*c4+c5] = build_long(lead, '0
                        '+c2, '0'+c3, '0'+c4, '0'+c5, '\n', '0', '0');

        for(unsigned long c3 = 1; c3 <= 9; ++c3)
            for(unsigned long c4 = 0; c4 <= 9; ++c4)
                for(unsigned long c5 = 0; c5 <= 9; ++c5)
```

```
        arr[100*c3+10*c4+c5] = build_long(lead, lead, '0'+c3,
            '0'+c4, '0'+c5, '\n', '0', '0');

        for(unsigned long c4 = 1; c4 <= 9; ++c4)
            for(unsigned long c5 = 0; c5 <= 9; ++c5)
                arr[10*c4+c5] = build_long(lead, lead, '0'+c4, '0
                '+c5, '\n', '0', '0');

        for(unsigned long c5 = 0; c5 <= 9; ++c5)
            arr[c5] = build_long(lead, lead, lead, lead, '0'+c5, '\n'
            , '0', '0');

    }

    array<unsigned long, 100'000> arr{};
};

constexpr output<true> out_lz;
constexpr output<false> out_ls;

void write_long(unsigned long l, unsigned long n) {
    *reinterpret_cast<unsigned long*> (buf_ptr) = l;
    buf_ptr += n;
}

pair<unsigned long, unsigned long> podziel(unsigned long a,
    unsigned long b) {
    return {a/b, a%b};
}

void write_int(unsigned long x) {
    if(x >= 1'000'000'000'000'000) {
        auto [abc, d] = podziel(x, 100'000);
        auto [ab, c] = podziel(abc, 100'000);
        auto [a, b] = podziel(ab, 100'000);
        write_long(out_ls.arr[a], 5);
        write_long(out_lz.arr[b], 5);
        write_long(out_lz.arr[c], 5);
        write_long(out_lz.arr[d], 6);
    } else if(x >= 10'000'000'000) {
        auto [ab, c] = podziel(x, 100'000);
        auto [a, b] = podziel(ab, 100'000);
        write_long(out_ls.arr[a], 5);
        write_long(out_lz.arr[b], 5);
        write_long(out_lz.arr[c], 6);
    }
    else if(x >= 100'000) {
        auto [a, b] = podziel(x, 100'000);
        write_long(out_ls.arr[a], 5);
        write_long(out_lz.arr[b], 6);
    } else {
        write_long(out_ls.arr[x], 6);
    }
}

void write_all() {
    //wypisuje wszystko, mozna wkleic na koniec maina
    write(STDOUT_FILENO, buf, buf_ptr-buf);
}
```

short-out.cpp

Opis: czas: ok. 1,47s na time, 1955ms na oiejq.

```
char buf[200 * 1'024 * 1'024];
char *buf_ptr = buf;

void write_char(char c) {
    *buf_ptr++ = c;
}
```

```
pair<unsigned long, unsigned long> podziel(unsigned long a,
    unsigned long b) {
    return {a/b, a%b};
}

void write_int_petla(unsigned long x) {
    do {
        auto [c, d] = podziel(x, 10);
        write_char('0'+d);
        x = c;
    } while (x > 0) ;
}

void write_int(unsigned long x) {
    char *old_buf_ptr = buf_ptr;
    write_int_petla(x);
    reverse(old_buf_ptr, buf_ptr);
    write_char('\n');
}

void write_all() {
    //wypisuje wszystko, mozna wkleic na koniec maina
    write(STDOUT_FILENO, buf, buf_ptr-buf);
}
```

no-buf-out.cpp

Opis: czas: ok. 0,67s na time, 2816ms na oiejq.

```
void write_int(unsigned long x) {
    char t[19];
    int i = 0;
    do {
        int d = x%10;
        t[i++] = '0'+d;
        x /= 10;
    } while (x > 0) ;

    while (--i >= 0) {
        putchar_unlocked(t[i]);
    }
    putchar_unlocked('\n');
}
```

3.2 Fastin

Jest kilka opcji. Różnią się długością (i złożonością kodu), bądź używaniem funkcji read() z bibliotekiunistd.h (gdyby był z nią problem to można użyć opcji bez tych funkcji).

Do każdego kodu dopisany jest przykładowy czas (wczytanie  $2 \cdot 10^7$  liczb). Rozmiary bufora zależą od rozmiaru wejścia (raczej mniejszy niż w poniższych kodach). Rozmiar powinien uwzględnić każdą cyfrę i spację. Jeżeli wejście jest za duże, to bufor może być mniejszy, a kod podzielony na kilka części oddzielonych flushowaniem (ważne by kod podzielić, **nie** dodawać ifa w środku żadnej funkcji, bo będzie wolno).

fastin.cpp

Opis: czas: 0,15s na time, 772ms na oiejq.

```
char buf[200 * 1'024 * 1'024];
int pos;

bool is_digit(char c) {
    return c >= '0';
}
```

```
}

int fastin(){
    int x = 0;
    char c;
    while (is_digit(c = buf[pos++])) {
        x = x * 10 + c - '0';
    }
    return x;
}

void read_all() {
    //wczytuje cale wejście, można wkleic na poczatku maina
    read(STDIN_FILENO, buf, 200 * 1'024 * 1'024);
}
```

no-buf-in.cpp

Opis: czas: 0,38s na time, 1052ms na oiejq.

```
bool is_digit(char c) {
    return c >= '0';
}

int fastin(){
    int x = 0;
    char c;
    while (is_digit(c = getchar_unlocked())) {
        x = x * 10 + c - '0';
    }
    return x;
}
```